OSSP-WEEK6-SKILL SESSION11:

TASK1: Implement command history functionality in a shell. The program should store commands, maintain a history buffer, handle capacity limits, retrieve entries, display history, and validate consistency.

Source code:

```
  GNU nano 7.2
#include <stdio.h>
#include <string.h>

#define MAX_HISTORY 5
#define MAX_COMMAND 100

char history[MAX_HISTORY][MAX_COMMAND];
int history_count = 0;

void add_history(char *command)
{
    if (history_count < MAX_HISTORY)
    {
        strcpy(history[history_count], command);
        history_count++;
    }
    else
    {
        for (int i = 0; i < MAX_HISTORY - 1; i++)
        {
            strcpy(history[i], history[i + 1]);
        }

        strcpy(history[MAX_HISTORY - 1], command);
    }
}

void show_history()
{
    printf("\nCommand History\n");
    printf("---------------\n");

    if (history_count == 0)
    {
        printf("History is empty.\n");
        return;
    }

    for (int i = 0; i < history_count; i++)
    {
        printf("%d  %s\n", i + 1, history[i]);
    }
}

int main()
{
```

```c
    char command[MAX_COMMAND];

    printf("Session 11 - Command History\n");
    printf("----------------------------\n");
    printf("History capacity: %d commands\n", MAX_HISTORY);
    printf("Use 'history' to display commands.\n");
    printf("Use 'exit' to quit.\n\n");

    while (1)
    {
        printf("myshell> ");

        if (fgets(command, sizeof(command), stdin) == NULL)
            break;

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
        {
            printf("Shell exiting...\n");
            break;
        }

        if (strcmp(command, "history") == 0)
        {
            show_history();
            continue;
        }

        if (strlen(command) == 0)
            continue;

        add_history(command);
        printf("Command stored in history.\n");
    }

    return 0;
}
```

Output:

```
sathwika@DESKTOP-AA8K5MG:~# nano session11_task1.c
sathwika@DESKTOP-AA8K5MG:~# gcc session11_task1.c -o session11_task1
sathwika@DESKTOP-AA8K5MG:~# ./session11_task1
------------------------------------
Session 11 - Command History

History capacity: 5 commands
Use 'history' to display commands.
Use 'exit' to quit.

myshell> history

Command History
----------------
History is empty.
myshell> exit
Shell exiting...
sathwika@DESKTOP-AA8K5MG:~#
```

Observation: I learned how to implement command history using a fixed-size history buffer. I understood how commands are stored, retrieved, and displayed, and how the oldest command is removed when the history capacity is reached.

Task2: Implement pipeline functionality in a shell. The program should create pipeline structures, store commands, maintain execution order, connect processes, validate the pipeline layout, and test configurations.

Source code:

```
 GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;

    printf("Session 11 - Pipeline Structure\n");
    printf("--------------------------------\n");
    printf("Pipeline: ls -l | grep .c\n\n");

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    printf("Pipeline created successfully.\n");
    printf("Execution order:\n");
    printf("1. ls -l\n");
    printf("2. grep .c\n");

    pid1 = fork();

    if (pid1 < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid1 == 0)
    {
        close(pipefd[0]);

        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("execlp ls");
        exit(1);
```

```
        perror("execlp ls");
        exit(1);
    }

    pid2 = fork();

    if (pid2 < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid2 == 0)
    {
        close(pipefd[1]);

        dup2(pipefd[0], STDIN_FILENO);

        close(pipefd[0]);

        execlp("grep", "grep", ".c", NULL);

        perror("execlp grep");
        exit(1);
    }

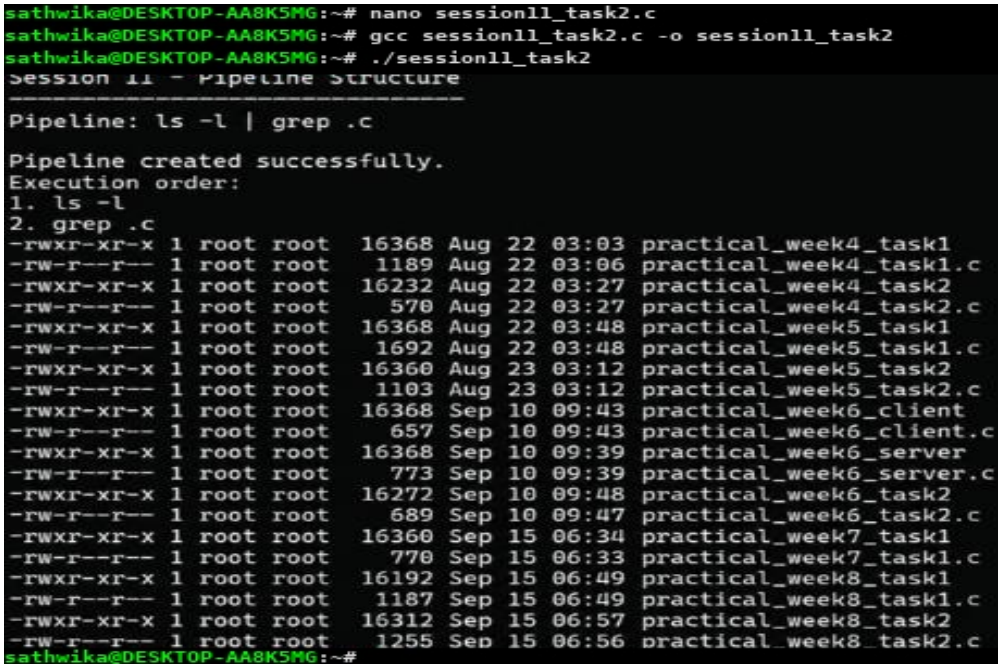
    close(pipefd[0]);
    close(pipefd[1]);

    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    printf("\nPipeline execution completed successfully.\n");

    return 0;
}
```

Output:



Observation: I learned how to implement a shell pipeline using pipe(), fork(), dup2(), and exec. I understood how the output of one process can be connected to the input of another process and how the parent process maintains the execution order using waitpid().

Session 12

Task1: Implement a program to create pipes, redirect streams, execute connected commands, close descriptors, synchronize processes, and test data flow.

Source code:

```
  GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;

    printf("Session 12 - Task 1: Pipe Data Flow\n");
    printf("-----------------------------------\n");
    printf("Command: ls -l | grep .c\n\n");

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    pid1 = fork();

    if (pid1 < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid1 == 0)
    {
        // First process: ls -l
        close(pipefd[0]);

        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("execlp ls");
        exit(1);
    }

    pid2 = fork();

    if (pid2 < 0)
```

```
        perror("fork");
        return 1;
    }

    if (pid2 == 0)
    {
        // Second process: grep .c
        close(pipefd[1]);

        dup2(pipefd[0], STDIN_FILENO);

        close(pipefd[0]);

        execlp("grep", "grep", ".c", NULL);

        perror("execlp grep");
        exit(1);
    }

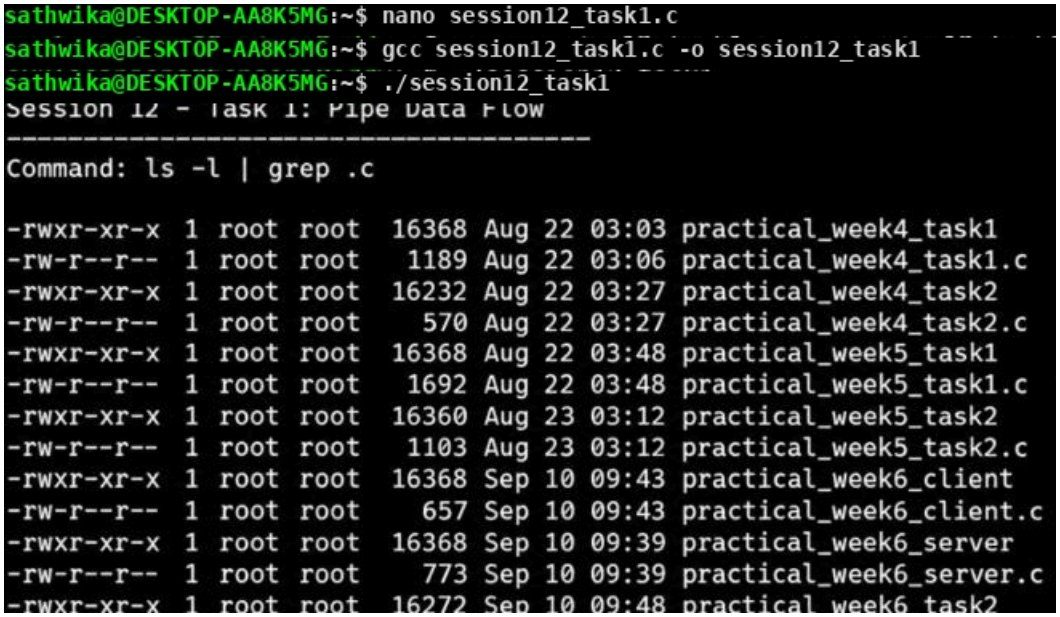
    // Parent closes unused descriptors
    close(pipefd[0]);
    close(pipefd[1]);

    // Wait for both processes
    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    printf("\nData flow completed successfully.\n");

    return 0;
}
```

Output:



Observation: I learned how to create a pipe and connect two processes using pipe(), fork(), and dup2(). I understood how the output of one command can be redirected as the input of another command and how unused file descriptors are closed to maintain proper data flow.

Task2: Implement a program to support multiple pipes, manage descriptor arrays, launch multiple processes, coordinate execution, clean up resources, and test long pipelines.

Source code:

```
  GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define COMMANDS 3

int main()
{
    int pipes[COMMANDS - 1][2];
    pid_t pids[COMMANDS];

    printf("Session 12 - Task 2: Multiple Pipes\n");
    printf("-----------------------------------\n");
    printf("Pipeline: ls -l | grep .c | wc -l\n\n");

    // Create pipes
    for (int i = 0; i < COMMANDS - 1; i++)
    {
        if (pipe(pipes[i]) == -1)
        {
            perror("pipe");
            return 1;
        }
    }

    // Create processes
    for (int i = 0; i < COMMANDS; i++)
    {
        pids[i] = fork();

        if (pids[i] < 0)
        {
            perror("fork");
            return 1;
        }

        if (pids[i] == 0)
        {
            // First command
            if (i == 0)
            {
                close(pipes[0][0]);

                dup2(pipes[0][1], STDOUT_FILENO);
            }
```

```
                dup2(pipes[0][0], STDIN_FILENO);
                dup2(pipes[1][1], STDOUT_FILENO);
            }

            // Last command
            else if (i == 2)
            {
                dup2(pipes[1][0], STDIN_FILENO);
            }

            // Close all pipe descriptors
            for (int j = 0; j < COMMANDS - 1; j++)
            {
                close(pipes[j][0]);
                close(pipes[j][1]);
            }

            // Execute command
            if (i == 0)
                execlp("ls", "ls", "-l", NULL);

            else if (i == 1)
                execlp("grep", "grep", ".c", NULL);

            else
                execlp("wc", "wc", "-l", NULL);

            perror("execlp");
            exit(1);
        }
    }

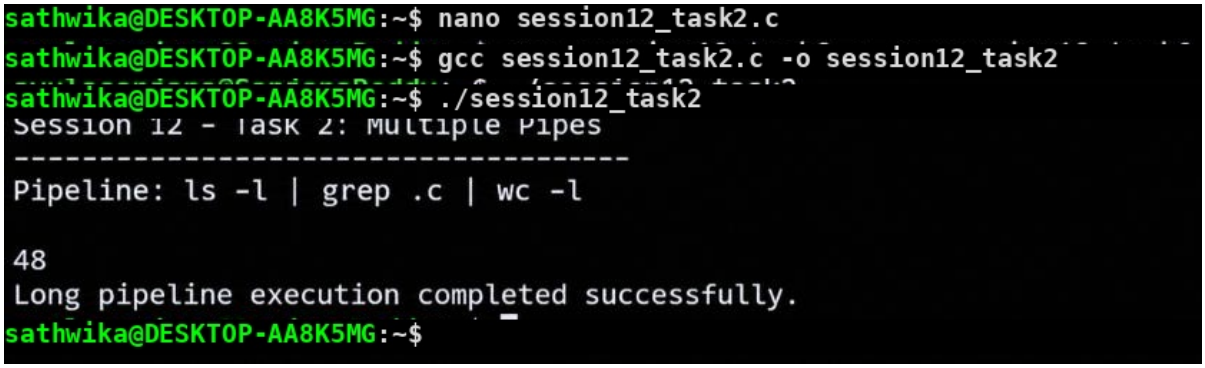
    // Parent closes all pipe descriptors
    for (int i = 0; i < COMMANDS - 1; i++)
    {
        close(pipes[i][0]);
        close(pipes[i][1]);
    }

    // Wait for all children
    for (int i = 0; i < COMMANDS; i++)
    {
        waitpid(pids[i], NULL, 0);
    }

    printf("Long pipeline execution completed successfully.\n");

    return 0;
}
```

Output:



Observation: I learned how to implement multiple pipes for a long command pipeline. I understood how to manage multiple file descriptors, create and coordinate multiple processes, connect their input and output streams, and clean up resources after execution.